

Financial University under the Government of Russian Federation

Nails

Semyon Babich, Damir Bekirov, Yury Korobkov

All contests

Season 2025-2026

1 Useful things

2 Data Structures

3 Mathematics

4 DP

5 Number Theory

6 Geometry

7 Graphs

8 Numerical

9 Strings

Useful things (1)

1.1 Template

```
template.cpp
27 lines

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;
typedef long double ld;

const int MAXN = 2e5 + 1;
const int MOD = 998244353;
const int INF = 2e9;
const ll INFLL = 1e18;

void solve () {

}

int32_t main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    int tests = 1;
    // cin >> tests;
    while (tests--) {
        solve();
    }
    return 0;
}
```

1.2 Stress-testing

```
dumbScript.py
56 lines

# Description: Stress testing script
# Both solutions need freopen("input.txt", "r", stdin)
# In tests.txt we generate tests in advance
# Specify how many lines are in the input in the
# $lines_of_input$ variable
# Specify how many tests we want to process in the $iterations$
# variable
```

```
import subprocess
import os
import shutil

shutil.copy('tests.txt', 'tmp_tests.txt')

cpp_file_a = 'a.cpp'
cpp_file_stupid = 'stupid.cpp'

lines_of_input = 2
iterations = 4

def run_program(command):
    result = subprocess.run(command, stdout=subprocess.PIPE,
                             stderr=subprocess.PIPE, text=True)
    return result.stdout.strip(), result.stderr.strip()

compile_result_a = run_program(['g++', cpp_file_a, '-o', 'a.out'])

compile_result_stupid = run_program(['g++', cpp_file_stupid, '-o', 'stupid.out'])

for _ in range(iterations):
    with open('tmp_tests.txt', 'r') as input_file:
        lines = input_file.readlines()

        if len(lines) < lines_of_input:
            print("Not enough lines for next test")
            break

    current_input = "".join(lines[:lines_of_input])

    with open('tmp_tests.txt', 'w') as input_file:
        input_file.writelines(lines[lines_of_input:])

    with open('input.txt', 'w') as input_file:
        input_file.writelines(lines[:lines_of_input])

    output_a, error_a = run_program(['./a.out'])

    output_stupid, error_stupid = run_program(['./stupid.out'])

    if output_a != output_stupid:
        print("Failed test:")
        print("Input:")
        print(current_input)
        print("output_stupid.txt:")
        print(output_stupid)
        print("output_a.txt:")
        print(output_a)
        break
    else:
        print("All good!")
```

1.3 pbds

```
pbds.hh
17 lines

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <bits/extc++.h>
using namespace __gnu_pbds;
typedef tree<int, null_type, less<int>, rb_tree_tag,
             tree_order_statistics_node_update> ordered_set;
// ordered_set X;
// *X.find_by_order(k) - kth order statistics
// X.order_of_key(n) - # elements that < n
typedef tree<string, null_type, trie_string_access_traits<>,
             pat_trie_tag, trie_prefix_search_node_update> pref_trie;
```

```
// pref_trie base;
// base.insert("hello") - add string to trie
// auto range = base.prefix_range(x.substr(1)); - [iter1, iter2) - strings with this prefix
struct chash { // large odd number for C
    const uint64_t C = 114e18 * acos(0) | 71;
    ll operator() (ll x) const { return __builtin_bswap64(x*C); }
};
gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

1.4 Pragmas

```
pragmas.hh
9 lines

// Description: O3 can greatly inflate the code, sometimes it is more profitable to change to O2
// Ofast is the strongest level
// avx2 may not be supported on Yandex.Contest, then avx or sse
// /2/3/4/4.1/4.2
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,bmt,lzcnt,popcnt")
// if it is necessary that the pragmas be applied strictly to one function
__attribute__((target("avx2"), optimize("O3", "unroll-loops")))
void work() {
    // do something
}
```

1.5 Randoms

```
randomIntegers.hh
5 lines

mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count()); // use current time as a seed
uniform_int_distribution<int> distrib(1, 10); // create distribution
int get_rand() {
    return distrib(rnd); // random from [1, 10]
}
```

Data Structures (2)

```
fenwick.hh
Description: Fenwick my twizz=)
22 lines

struct FT {
    vector<ll> s;
    FT(int n) : s(n) {}
    void update(int pos, ll dif) { // a[pos] += dif
        for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
    }
    ll query(int pos) { // sum of values in [0, pos]
        ll res = 0;
        for (; pos > 0; pos &= pos - 1) res += s[pos-1];
        return res;
    }
    int lower_bound(ll sum) { // min pos st sum of [0, pos] >= sum
        // Returns n if no sum is >= sum, or -1 if empty sum is.
        if (sum <= 0) return -1;
        int pos = 0;
        for (int pw = 1 << 25; pw; pw >= 1) {
            if (pos + pw <= sz(s) && s[pos + pw-1] < sum)
                pos += pw, sum -= s[pos-1];
        }
        return pos;
    }
};
```

segmentTree.hh

Description: Zero-indexed max-tree. Bounds are inclusive to the left and exclusive to the right. Can be changed by modifying T, f and unit.

Time: $\mathcal{O}(\log N)$

```
19 lines
struct Tree {
    typedef int T;
    static constexpr T unit = INT_MIN;
    T f(T a, T b) { return max(a, b); } // (any associative fn)
    vector<T> s; int n;
    Tree(int n = 0, T def = unit) : s(2*n, def), n(n) {}
    void update(int pos, T val) {
        for (s[pos += n] = val; pos /= 2;)
            s[pos] = f(s[pos * 2], s[pos * 2 + 1]);
    }
    T query(int b, int e) { // query [b, e)
        T ra = unit, rb = unit;
        for (b += n, e += n; b < e; b /= 2, e /= 2) {
            if (b % 2) ra = f(ra, s[b++]);
            if (e % 2) rb = f(s[--e], rb);
        }
        return f(ra, rb);
    }
};
```

sparseTable.hh

Description: Geek from Tyumen Region thinks that he is RMQ Data Structure

```
27 lines
template <typename T>
struct SparseTable{
public:
    SparseTable (vector<T> &a) : n(a.size()), a(a) {
        init(n);
    }
    T rmq(T l, T r) {
        T t = __lg(r - l);
        return min(g[t][l], g[t][r - (1 << t)]);
    }
private:
    int n;
    vector<T> &a;
    vector<vector<T>> g;
    void init(int n) {
        int logn = __lg(n);
        g.assign(logn + 1, vector<T>(n));
        for (int i = 0; i < n; ++i) {
            g[0][i] = a[i];
        }
        for (int l = 0; l <= logn - 1; l++) {
            for (int i = 0; i + (2 << l) <= n; i++) {
                g[l + 1][i] = min(g[l][i], g[l][i + (1 << l)]);
            }
        }
    }
};
```

treap.hh

Description: Just treap=)

```
55 lines
struct Node {
    Node *l = 0, *r = 0;
    int val, y, c = 1;
    Node(int val) : val(val), y(rand()) {}
    void recalc();
};

int cnt(Node* n) { return n ? n->c : 0; }
void Node::recalc() { c = cnt(l) + cnt(r) + 1; }
```

```
template<class F> void each(Node* n, F f) {
    if (n) { each(n->l, f); f(n->val); each(n->r, f); }
}

pair<Node*, Node*> split(Node* n, int k) {
    if (!n) return {};
    if (cnt(n->l) >= k) { // "n->val >= k" for lower_bound(k)
        auto pa = split(n->l, k);
        n->l = pa.second;
        n->recalc();
        return {pa.first, n};
    } else {
        auto pa = split(n->r, k - cnt(n->l) - 1); // and just "k"
        n->r = pa.first;
        n->recalc();
        return {n, pa.second};
    }
}

Node* merge(Node* l, Node* r) {
    if (!l) return r;
    if (!r) return l;
    if (l->y > r->y) {
        l->r = merge(l->r, r);
        l->recalc();
        return l;
    } else {
        r->l = merge(l, r->l);
        r->recalc();
        return r;
    }
}

Node* ins(Node* t, Node* n, int pos) {
    auto [l,r] = split(t, pos);
    return merge(merge(l, n), r);
}

// Example application: move the range [l, r) to index k
void move(Node& t, int l, int r, int k) {
    Node *a, *b, *c;
    tie(a,b) = split(t, l); tie(b,c) = split(b, r - l);
    if (k <= l) t = merge(ins(a, b, k), c);
    else t = merge(a, ins(c, b, k - r));
}

lazySegmentTree.hh
Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals.
Usage: Node* tr = new Node(v, 0, sz(v));
Time: O(log N).
50 lines

const int inf = 1e9;
struct Node {
    Node *l = 0, *r = 0;
    int lo, hi, mset = inf, madd = 0, val = -inf;
    Node(int lo,int hi) : lo(lo), hi(hi) {} // Large interval of -inf
    Node(vector<int>& v, int lo, int hi) : lo(lo), hi(hi) {
        if (lo + 1 < hi) {
            int mid = lo + (hi - lo)/2;
            l = new Node(v, lo, mid); r = new Node(v, mid, hi);
            val = max(l->val, r->val);
        }
        else val = v[lo];
    }
    int query(int L, int R) {
        if (R <= lo || hi <= L) return -inf;
        if (L <= lo && hi <= R) return val;
```

```
        push();
        return max(l->query(L, R), r->query(L, R));
    }
    void set(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) mset = val = x, madd = 0;
        else {
            push(), l->set(L, R, x), r->set(L, R, x);
            val = max(l->val, r->val);
        }
    }
    void add(int L, int R, int x) {
        if (R <= lo || hi <= L) return;
        if (L <= lo && hi <= R) {
            if (mset != inf) mset += x;
            else madd += x;
            val += x;
        }
        else {
            push(), l->add(L, R, x), r->add(L, R, x);
            val = max(l->val, r->val);
        }
    }
    void push() {
        if (!l) {
            int mid = lo + (hi - lo)/2;
            l = new Node(lo, mid); r = new Node(mid, hi);
        }
        if (mset != inf)
            l->set(lo, hi, mset), r->set(lo, hi, mset), mset = inf;
        else if (madd)
            l->add(lo, hi, madd), r->add(lo, hi, madd), madd = 0;
    }
};

persistentDSU.hh
Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: int t = uf.time(); ...; uf.rollback(t);
Time: O(log(N))
21 lines

struct DSU {
    vector<int> e; vector<pair<int, int>> st;
    DSU (int n) : e(n, -1) {}
    int size(int x) { return -e[find(x)]; }
    int find(int x) { return e[x] < 0 ? x : find(e[x]); }
    int time() { return sz(st); }
    void rollback(int t) {
        for (int i = time(); i --> t;)
            e[st[i].first] = st[i].second;
        st.resize(t);
    }
    bool join(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (e[a] > e[b]) swap(a, b);
        st.push_back({a, e[a]});
        st.push_back({b, e[b]});
        e[a] += e[b]; e[b] = a;
        return true;
    }
};

intervalContainer.hh
Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
Time: O(log N)
25 lines
```

```
using pii = pair<int, int>;

set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
    if (L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while (it != is.end() && it->first <= R) {
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if (it != is.begin() && (--it)->second >= L) {
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L, R});
}

void removeInterval(set<pii>& is, int L, int R) {
    if (L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if (it->first == L) is.erase(it);
    else (int&)it->second = L;
    if (R != r2) is.emplace(R, r2);
}
```

Mathematics (3)

3.1 Master Theorem

$$T(n) = \begin{cases} aT(\frac{n}{b}) + \Theta(n^c) & n > n_0 \\ \Theta(1) & n \leq n_0 \end{cases}$$

- $c > \log_b a : T(n) = \Theta(n^c)$
- $c = \log_b a : T(n) = \Theta(n^c \log n)$
- $c < \log_b a : T(n) = \Theta(n^{\log_b a})$

3.2 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e & x &= \frac{ed - bf}{ad - bc} \\ cx + dy &= f & \Rightarrow y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

3.3 Recurrences

If $a_n = c_1a_{n-1} + \dots + c_ka_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1r_1^n + \dots + d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g.
 $a_n = (d_1n + d_2)r^n$.

3.4 Trigonometry

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \end{aligned}$$

$$\begin{aligned} \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

3.5 Geometry

3.5.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):

$$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

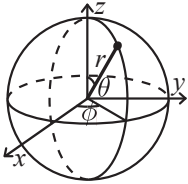
3.5.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2f^2 - F^2}$$

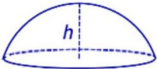
For cyclic quadrilaterals the sum of opposite angles is 180° , $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

3.5.3 Spherical coordinates

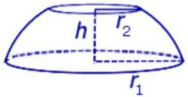


$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \arccos(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \text{atan2}(y, x) \end{aligned}$$

3.5.4 Random facts for Yura Korobkov



Volume of sphere segment: $V = \pi h^2(r - \frac{1}{3}h)$



Volume of sphere layer: $V = \frac{1}{6}\pi h^3 + \frac{1}{2}\pi h(r_1^2 + r_2^2)$

3.6 Derivatives/Integrals

$$\frac{d}{dx} \arcsin x = \frac{1}{\sqrt{1 - x^2}} \quad \frac{d}{dx} \arccos x = -\frac{1}{\sqrt{1 - x^2}}$$

$$\frac{d}{dx} \tan x = 1 + \tan^2 x \quad \frac{d}{dx} \arctan x = \frac{1}{1 + x^2}$$

$$\int \tan ax = -\frac{\ln |\cos ax|}{a} \quad \int x \sin ax = \frac{\sin ax - ax \cos ax}{a^2}$$

$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2} \text{erf}(x) \quad \int x e^{ax} dx = \frac{e^{ax}}{a^2} (ax - 1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

3.7 Sums

$$c^a+c^{a+1}+\cdots+c^b=\frac{c^{b+1}-c^a}{c-1}, c\neq 1$$

$$\begin{aligned}1+2+3+\cdots+n&=\frac{n(n+1)}{2}\\1^2+2^2+3^2+\cdots+n^2&=\frac{n(2n+1)(n+1)}{6}\\1^3+2^3+3^3+\cdots+n^3&=\frac{n^2(n+1)^2}{4}\\1^4+2^4+3^4+\cdots+n^4&=\frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}\end{aligned}$$

3.8 Series

$$e^x=1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ldots, (-\infty < x < \infty)$$

$$\ln(1+x)=x-\frac{x^2}{2}+\frac{x^3}{3}-\frac{x^4}{4}+\ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x}=1+\frac{x}{2}-\frac{x^2}{8}+\frac{2x^3}{32}-\frac{5x^4}{128}+\ldots, (-1 \leq x \leq 1)$$

$$\sin x=x-\frac{x^3}{3!}+\frac{x^5}{5!}-\frac{x^7}{7!}+\ldots, (-\infty < x < \infty)$$

$$\cos x=1-\frac{x^2}{2!}+\frac{x^4}{4!}-\frac{x^6}{6!}+\ldots, (-\infty < x < \infty)$$

3.9 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu=\mathbb{E}(X)=\sum_x xp_X(x)$ and variance $\sigma^2=V(X)=\mathbb{E}(X^2)-(\mathbb{E}(X))^2=\sum_x (x-\mathbb{E}(X))^2p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX+bY)=a\mathbb{E}(X)+b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX+bY)=a^2V(X)+b^2V(Y).$$

3.10 Generating functions

Example of finding generating functions:

$$a_0=1$$

$$a_1=2$$

$$a_n=6a_{n-1}-8a_{n-2}+n, n\geq 2$$

$$G(z)=a_0+a_1z+\sum_{n=2}^\infty (6a_{n-1}-8a_{n-2}+n)z^n$$

$$G(z)=a_0+a_1z+6\sum_{n=2}^\infty a_{n-1}z^n-8\sum_{n=2}^\infty a_{n-2}z^n+\sum_{n=2}^\infty nz^n$$

$$G(z)=a_0+a_1z+6z\sum_{n=1}^\infty a_nz^n-8z^2\sum_{n=0}^\infty a_nz^n+\sum_{n=2}^\infty nz^n$$

$$G(z)=a_0+a_1z+6z(G(z)-a_0)-8z^2G(z)+\sum_{n=2}^\infty nz^n$$

$$G(z)=1-4z+6zG(z)-8z^2G(z)+\sum_{n=2}^\infty nz^n$$

To calculate $\sum_{n=2}^\infty nz^n$ use generating function $B(z)$ for $b_n=(1,1,1,1,\dots)$:

$$zB'(z)=z(\sum_{n=0}^\infty b_nz^n)'=\sum_{n=0}^\infty b_nz^n$$

Then we can do:

$$\sum_{n=2}^\infty nz^2=z\sum_{n=2}^\infty nz^{n-1}=z(\sum_{n=2}^\infty z^n)'$$

$$\sum_{n=2}^\infty z^n=\sum_{n=0}^\infty z^n-1-z=\frac{1}{1-z}-1-z=\frac{z^2}{1-z}$$

$$z(\frac{z^2}{1-z})'=\frac{z^2(2-z)}{(1-z)^2}$$

Then:

$$G(z)=1-4z+6zG(z)-8z^22G(z)+\frac{z^2(2-z)}{(1-z)^2}$$

$$G(z)=\frac{1-6z+11z^2-5z^3}{(1-6z+8z^2)(1-z)^2}$$

Последовательность	Производящие функция в виде ряда	Производящие функция в замкнутом виде
(1, 0, 0, ...)	1	1
(0, 0, ..., 0, 1, 0, 0, ...) (от нуля и началу)	z^n	z^n
(1, 1, 1, ...)	$\sum z^n$	$\frac{1}{1-z}$
(1, 0, 0, ..., 0, 1, 0, 0, ..., 0, 1, 0, 0, ...) (повторяется через m)	$\sum z^{nm}$	$\frac{1}{1-z^n}$
(1, -1, 1, -1, ...)	$\sum (-1)^n z^n$	$\frac{1}{1+z}$
(1, 2, 3, 4, ...)	$\sum (n+1) z^n$	$\frac{1}{(1-z)^2}$
(1, 2, 4, 8, 16, ...)	$\sum 2^n z^n$	$\frac{1}{(1-2z)}$
(1, r, r^2, r^3, ...)	$\sum r^n z^n$	$\frac{1}{(1-rz)}$
$(\binom{m}{n}, \binom{m}{n-1}, \binom{m}{n-2}, \binom{m}{n-3}, \dots)$	$\sum \binom{m}{n} z^n$	$(1+z)^m$
$(1, \binom{m}{n}, \binom{m+1}{n}, \binom{m+2}{n}, \dots)$	$\sum \binom{m+n-1}{n} z^n$	$\frac{1}{(1-z)^{m+1}}$
$(0, 1, -\frac{1}{2}, \frac{1}{3}, -\frac{1}{4}, \dots)$	$\sum \frac{(-1)^{n+1}}{n} z^n$	$\ln(1+z)$
$(1, 1, \frac{1}{2}, \frac{1}{6}, \frac{1}{24}, \dots)$	$\sum \frac{1}{n!} z^n$	e^z
$(1, -\frac{1}{2!}m^2, \frac{1}{4!}m^4, -\frac{1}{6!}m^6, \frac{1}{8!}m^8, \dots)$	$\sum \frac{1}{(2n)!} m^{(2n)}$	$\cos m$
$(m, -\frac{1}{3!}m^3, \frac{1}{5!}m^5, -\frac{1}{7!}m^7, \frac{1}{9!}m^9, \dots)$	$\sum \frac{1}{(2n-1)!} m^{(2n-1)}$	$\sin m$

DP (4)

4.1 Layer DP Optimization

divideAndConquer.hh

Description: when $opt[i][j] \leq opt[i][j+1]$

Time: $\mathcal{O}(kn \log n)$

13 lines

```
void calc(int tl, int tr, int l, int r, int layer){
    if(tl > tr) return;
    int mid = (tl + tr) >> 1;
    int opt = -1;
    for (int i = l; i <= min(r, mid - 1); ++i) {
        if (dp[mid][layer + 1] > dp[i][layer] + cost[i + 1][mid
        ]) {
            opt = i;
            dp[mid][layer + 1] = dp[i][layer] + cost[i + 1][mid
            ];
        }
    }
    calc(tl, mid - 1, l, opt, layer);
    calc(mid + 1, tr, opt, r, layer);
}
```

knuthOptimization.hh

Description: when $opt[i-1][j] \leq opt[i][j] \leq opt[i][j+1]$

Time: $\mathcal{O}(n^2)$

11 lines

```
for (int len = 2; len <= n; ++len) {
    for (int l = 0; l <= n - len; ++l) {
        int r = l + len;
        for (int i = cut[l][r - 1]; i <= cut[l + 1][r]; ++i) {
            if (dp[l][r] > cost[l + 1][i] + dp[l][i] + cost[i +
            2][r] + dp[i + 1][r]) {
                cut[l][r] = i;
                dp[l][r] = cost[l + 1][i] + dp[l][i] + cost[i +
                2][r] + dp[i + 1][r];
            }
        }
    }
}
```

convexHullTrick.hh

Description: Use this if your dp transformable to: $dp[i][m] = \min(a[k] * x + b[k])$
E.g. $f[i][j] = \min(f[k][j-1] + (x_{i-1} - x_k)^2)$
 $f[i][j] = \min(f[k][j-1] + x_k^2 - 2x_kx_{i-1}) + x_{i-1}^2$
 $a[k] = f[k][j-1] + x_k^2, b[k] = -2x_k$

Time: $\mathcal{O}(nk)$	35 lines
<pre>struct Line { mutable ll k, m, p; // p is the position from which the // line is optimal ll val (ll x) const { return k * x + m; } bool operator< (const Line& o) const { return p < o.p; } }; ll floordiv (ll a, ll b) { return a / b - ((a*b) < 0 && a % b); } // queries and line intersections should be in range (-INF, INF) const ll INF = 1e17; struct LineContainer : vector<Line> { ll isect(const Line& a, const Line& b) { if (a.k == b.k) return a.m > b.m ? (-INF) : INF; ll res = floordiv(b.m - a.m, a.k - b.k); if (a.val(res) < b.val(res)) res++; return res; } void add(ll k, ll m) { Line a = {k,m,INF}; while (!empty() && isect(a, back()) <= back().p) pop_back(); a.p = empty() ? (-INF) : isect(a, back()); push_back(a); } ll query(ll x) { assert(!empty()); return (--upper_bound(begin(), end(), Line({0, 0, x}))) ->val(x); } int qi = 0; ll sorted_query(ll x) { assert(!empty()); qi = min(qi, (int)size() - 1); while(qi < size() - 1 && (*this)[qi + 1].p <= x) qi++; return (*this)[qi].val(x); } };</pre>	

liChaoTree.hh

Description: good with lazySegmentTree.hh idea sometimes. It's just got to be here. Don't touch it.

	49 lines
<pre>struct LiChaoTree{ struct line { int k = 0, m = 0; line() {} line(int k, int m): k(k), m(m) {} int get(int x) { return k * x + m; } }; public: LiChaoTree (int _maxn) : maxn(_maxn) { t.assign(4 * maxn); } void upd(int v = 1, int tl = 0, int tr = maxn - 1, line L) { if (tl > tr) { return; } int tm = (tl + tr) / 2; bool l = L.get(tl) > t[v].get(tl); bool mid = L.get(tm) > t[v].get(tm); if (mid) { swap(L, t[v]); } }</pre>	

<pre> if (l != mid) { upd(2 * v, tl, tm - 1, L); } else { upd(2 * v + 1, tm + 1, tr, L); } } int get(int v = 1, int tl = 0, int tr = maxn - 1, int x) { if (tl > tr) { return 0; } int tm = (tl + tr) / 2; if (x == tm) { return t[v].get(x); } if (x < tm) { return max(t[v].get(x), get(2 * v, tl, tm - 1, x)); } else { return max(t[v].get(x), get(2 * v + 1, tm + 1, tr, x)); } } } private: int maxn; vector <line> t; }</pre>	
--	--

lambdaOptimization.hh

Description: Transforming CHT to: to $dp[i] = \min(dp[j] + cost(j+1, i)) + \lambda$ Using binary search for λ to find first that best number of segments is exactly k

Time: $\mathcal{O}(n \log n)$	31 lines
<pre>void init() { for (int i = 0; i < maxn; i++) { dp[i] = make_pair(1, 0); } } pair <ll, int> check(ll x) { // change this to CHT init(); dp[0] = make_pair(0, 1); // 1-indexation for (int i = 1; i <= n; i++) { for (int j = 0; j < i; j++) { dp[i] = min(dp[i], {dp[j].first + cost[j + 1][i] + x, dp[j].second + 1}); } } return dp[n]; } ll solve() { ll l = -1e14; ll r = 1; while (l + 1 < r) { ll mid = (l + r) / 2; pair<ll, int> x = check(mid); if (x.second >= k) { l = mid; } else { r = mid; } } pair<ll, int> result = check(1); return result.first - 1 * result.second; }</pre>	

4.2 Simulated Annealing

simulatedAnnealing.hh

Description: Use it if you chill guy =)
There is solution of a problem of placing 8 queens on a chessboard in a such way that they don't attack each other
 $f(p)$ - number of the queens that don't attack anothers

	40 lines
<pre>const int n = 100; const int k = 1000; int f(vector<int> &p) { int s = 0; for (int i = 0; i < n; i++) { int d = 1; for (int j = 0; j < i; j++) if (abs(i - j) == abs(p[i] - p[j])) d = 0; s += d; } return s; } double rnd() { return double(rand()) / RAND_MAX; } int main() { vector<int> v(n); iota(v.begin(), v.end(), 0); shuffle(v.begin(), v.end()); // generate initial // permutation int ans = f(v); double t = 1; for (int i = 0; i < k && ans < n; i++) { t *= 0.99; vector<int> u = v; swap(u[rand() % n], u[rand() % n]); int val = f(u); if (val > ans rnd() < exp((val - ans) / t)) { v = u; ans = val; } } for (int x : v) cout << x + 1 << " "; return 0; }</pre>	

Number Theory (5)

bitGCD.hh

Description: Fast GCD
Time: Can speed up `_gcd` by 1.7x

	19 lines
<pre>unsigned int gcd (unsigned int u, unsigned int v) { int shift, uz, vz; if (u == 0) return v; if (v == 0) return u; uz = __builtin_ctz(u); vz = __builtin_ctz(v); shift = uz > vz ? vz : uz; u >>= uz; do { v >>= vz; int diff = v; diff == u; vz = __builtin_ctz(diff); }</pre>	

```
        if (diff == 0) break;
        if (v < u) u = v;
        v = abs(diff);
    } while (1);
    return u << shift;
}
```

euclid.hh

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `_gcd` instead. If a and b are coprime, then x is the inverse of a (mod b).

```
5 lines
11 euclid(11 a, 11 b, 11 &x, 11 &y) {
    if (!b) return x = 1, y = 0, a;
    11 d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

factor.hh

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

```
"MillerRabin.hh" 18 lines
ull pollard(ull n) {
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x) { return modmul(x, x, n) + i; };
    while (t++ % 40 || __gcd(prd, n) == 1) {
        if (x == y) x = ++i, y = f(x);
        if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}
vector<ull> factor(ull n) {
    if (n == 1) return {};
    if (isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

fastEratosthenes.hh

Description: Prime sieve for generating all primes smaller than LIM.

Time: LIM=1e9 $\approx$ 1.5s

```
20 lines
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1));
    vector<pii> cp;
    for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
        cp.push_back({i, i * i / 2});
        for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
    for (int L = 1; L <= R; L += S) {
        array<bool, S> block{};
        for (auto &[p, idx] : cp)
            for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
        rep(i,0,min(S, R - L))
            if (!block[i]) pr.push_back((L + i) * 2 + 1);
    }
    for (int i : pr) isPrime[i] = 1;
    return pr;
}
```

millerRabin.hh

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \bmod c$.

```
12 lines
bool isPrime(ull n) {
    if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n-1), d = n >> s;
    for (ull a : A) { // ^ count trailing zeroes
        ull p = modpow(a%n, d, n), i = s;
        while (p != 1 && p != n - 1 && a % n && i--)
            p = modmul(p, p, n);
        if (p != n-1 && i != s) return 0;
    }
    return 1;
}
```

modLog.hh

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. `modLog(a,1,m)` can be used to calculate the order of a .

Time: $\mathcal{O}\left(\sqrt{m}\right)$

```
11 lines
11 modLog(11 a, 11 b, 11 m) {
    11 n = (11) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<11, 11> A;
    while (j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if (e == b % m) return j;
    if (__gcd(m, e) == __gcd(m, b))
        rep(i,2,n+2) if (A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

modMulLL.hh

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$.

Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow

```
11 lines
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
    11 ret = a * b - M * ull(1.L / M * a * b);
    return ret + M * (ret < 0) - M * (ret >= (11)M);
}
ull modpow(ull b, ull e, ull mod) {
    ull ans = 1;
    for (; e; b = modmul(b, b, mod), e /= 2)
        if (e & 1) ans = modmul(ans, b, mod);
    return ans;
}
```

modPow.hh

const 11 mod = 1000000007; // faster if const

```
8 lines
11 modpow(11 b, 11 e) {
    11 ans = 1;
    for (; e; b = b * b % mod, e /= 2)
        if (e & 1) ans = ans * b % mod;
    return ans;
}
```

modSqrt.hh

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

```
24 lines
"ModPow.h"
11 sqrt(11 a, 11 p) {
```

```
a %= p; if (a < 0) a += p;
if (a == 0) return 0;
assert(modpow(a, (p-1)/2, p) == 1); // else no solution
if (p % 4 == 3) return modpow(a, (p+1)/4, p);
// a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
11 s = p - 1, n = 2;
int r = 0, m;
while (s % 2 == 0)
    ++r, s /= 2;
while (modpow(n, (p - 1) / 2, p) != p - 1) ++n;
11 x = modpow(a, (s + 1) / 2, p);
11 b = modpow(a, s, p), g = modpow(n, s, p);
for (; r = m) {
    11 t = b;
    for (m = 0; m < r && t != 1; ++m)
        t = t * t % p;
    if (m == 0) return x;
    11 gs = modpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
}
}
```

phiCalc.hh

Description: Euler's ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p - 1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$. $\sum_{d|n} \phi(d) = n$, $\sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$

Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.

Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \forall a$.

```
8 lines
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
    rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
    for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for (int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

Geometry (6)

6.1 Geometric primitives

Point.h

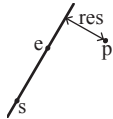
Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

```
28 lines
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y) {}
    bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
    bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
```

```
// angle to x-axis in interval [-pi, pi]
double angle() const { return atan2(y, x); }
P unit() const { return *this/dist(); } // makes dist()==1
P perp() const { return P(-y, x); } // rotates +90 degrees
P normal() const { return perp().unit(); }
// returns point rotated 'a' radians ccw around the origin
P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); }
friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << ", " << p.y << ")"; }
};
```

lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.



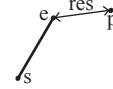
```
"Point.h"
4 lines

template<class P>
double lineDist(const P& a, const P& b, const P& p) {
    return (double) (b-a).cross(p-a) / (b-a).dist();
}
```

SegmentDistance.h

Description:
Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;



```
"Point.h"
6 lines

typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
    if (s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d,max(.0, (p-s).dot(e-s)));
    return ((p-s)*d-(e-s)*t).dist()/d;
}
```

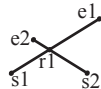
SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;

```
"Point.h", "OnSegment.h"
13 lines

template<class P> vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if (onSegment(c, d, a)) s.insert(a);
    if (onSegment(c, d, b)) s.insert(b);
    if (onSegment(a, b, c)) s.insert(c);
    if (onSegment(a, b, d)) s.insert(d);
```



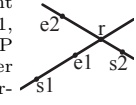
```
return {all(s)};
}
```

lineIntersection.h
Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.

Usage: auto res = lineInter(s1,e1,s2,e2);
if (res.first == 1)
cout << "intersection point at " << res.second << endl;

```
"Point.h"
8 lines

template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0) // if parallel
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```



sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 ⇔ left/on line/right. If the optional argument eps is given 0 is returned if p is within distance eps from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: bool left = sideOf(p1,p2,q)==1;

```
"Point.h"
9 lines

template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }
```

```
template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
    auto a = (e-s).cross(p-s);
    double l = (e-s).dist()*eps;
    return (a > l) - (a < -l);
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

```
"Point.h"
3 lines

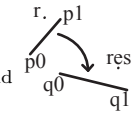
template<class P> bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

linearTransformation.h

Description:
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.

```
"Point.h"
6 lines

typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
    P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
}
```



Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

```
35 lines

struct Angle {
    int x, y;
    int t;
    Angle(int x, int y, int t=0) : x(x), y(y), t(t) {}
    Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
    Angle t180() const { return {-x, -y, t + half()}; }
    Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
    // add a.dist2() and b.dist2() to also compare distances
    return make_tuple(a.t, a.half(), a.y * (ll)b.x) <
        make_tuple(b.t, b.half(), a.x * (ll)b.y);
}
```

```
// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return (b < a.t180() ?
        make_pair(a, b) : make_pair(b, a.t360()));
}
Angle operator+(Angle a, Angle b) { // point a + vector b
    Angle r(a.x + b.x, a.y + b.y, a.t);
    if (a.t180() < r) r.t--;
    return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
    int tu = b.t - a.t; a.t = b.t;
    return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
}
```

6.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

```
"Point.h"
11 lines

typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
    if (a == b) { assert(r1 != r2); return false; }
    P vec = b - a;
    double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
        p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
    if (sum*sum < d2 || dif*dif > d2) return false;
    P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
    *out = {mid + per, mid - per};
    return true;
}
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

```
"Point.h" 13 lines

template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1, 1}) {
        P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

CirclePolygonIntersection.h

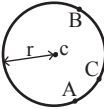
Description: Returns the area of the intersection of a circle with a ccw polygon.

```
".../geometry/Point.h" 19 lines

typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
    auto tri = [&](P p, P q) {
        auto r2 = r * r / 2;
        P d = q - p;
        auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
        auto det = a * a - b;
        if (det <= 0) return arg(p, q) * r2;
        auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
        if (t < 0 || 1 <= s) return arg(p, q) * r2;
        P u = p + d * s, v = p + d * t;
        return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
    };
    auto sum = 0.0;
    rep(i,0,sz(ps))
        sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
    return sum;
}
```

circumcircle.h

Description: The circumcircle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



```
"Point.h" 9 lines

typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
    return (B-A).dist()*(C-B).dist()*(A-C).dist() /
        abs((B-A).cross(C-A))/2;
}

P ccCenter(const P& A, const P& B, const P& C) {
    P b = C-A, c = B-A;
    return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.

Time: expected $\mathcal{O}(n)$

```
"circumcircle.h" 17 lines

pair<P, double> mec(vector<P> ps) {
    shuffle(all(ps), mt19937(time(0)));
    P o = ps[0];
    double r = 0, EPS = 1 + 1e-8;
    rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
        o = ps[i], r = 0;
        rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
            o = (ps[i] + ps[j]) / 2;
            r = (o - ps[i]).dist();
            rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
                o = ccCenter(ps[i], ps[j], ps[k]);
                r = (o - ps[i]).dist();
            }
        }
    }
    return {o, r};
}
```

6.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

```
Usage: vector<P> v = {P{4,4}, P{1,2}, P{2,1}};
bool in = inPolygon(v, P{3, 3}, false);
Time:  $\mathcal{O}(n)$ 

"Point.h", "OnSegment.h", "SegmentDistance.h" 11 lines

template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
    int cnt = 0, n = sz(p);
    rep(i,0,n) {
        P q = p[(i + 1) % n];
        if (onSegment(p[i], q, a)) return !strict;
        //or: if (segDist(p[i], q, a) <= eps) return !strict;
        cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
    }
    return cnt;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

```
"Point.h" 6 lines

template<class T>
T polygonArea2(vector<Point<T>>& v) {
    T a = v.back().cross(v[0]);
    rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
    return a;
}
```

PolygonCenter.h

Description: Returns the center of mass for a polygon.

```
Time:  $\mathcal{O}(n)$ 

"Point.h" 9 lines

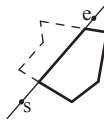
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
    P res(0, 0); double A = 0;
    for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
        res = res + (v[i] + v[j]) * v[j].cross(v[i]);
        A += v[j].cross(v[i]);
    }
    return res / A / 3;
}
```

PolygonCut.h

```
Description:
Returns a vector with the vertices of a polygon with every-
thing to the left of the line going from s to e cut away.
Usage: vector<P> p = ...;
p = polygonCut(p, P(0,0), P(1,0));

"Point.h", "lineIntersection.h" 13 lines

typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
    vector<P> res;
    rep(i,0,sz(poly)) {
        P cur = poly[i], prev = i ? poly[i-1] : poly.back();
        bool side = s.cross(e, cur) < 0;
        if (side != (s.cross(e, prev) < 0))
            res.push_back(lineInter(s, e, cur, prev).second);
        if (side)
            res.push_back(cur);
    }
    return res;
}
```



ConvexHull.h

Description: Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull. **Time:** $\mathcal{O}(n \log n)$

```
"Point.h" 13 lines

typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
    if (sz(pts) <= 1) return pts;
    sort(all(pts));
    vector<P> h(sz(pts)+1);
    int s = 0, t = 0;
    for (int it = 2; it--; s = --t, reverse(all(pts)))
        for (P p : pts) {
            while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
            h[t++] = p;
        }
    return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
}
```



HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

```
Time:  $\mathcal{O}(n)$ 

"Point.h" 12 lines

typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
    int n = sz(S), j = n < 2 ? 0 : 1;
    pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
    rep(i,0,j)
        for (; j = (j + 1) % n) {
            res = max(res, {S[i] - S[j]}.dist2(), {S[i], S[j]});
            if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
                break;
        }
    return res.second;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

```
Time:  $\mathcal{O}(\log N)$ 

"Point.h", "sideOf.h", "OnSegment.h" 14 lines

typedef Point<ll> P;
```

```
bool inHull(const vector<P>& l, P p, bool strict = true) {
    int a = 1, b = sz(l) - 1, r = !strict;
    if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
    if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
    if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p) <= -r)
        return false;
    while (abs(a - b) > 1) {
        int c = (a + b) / 2;
        (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
    }
    return sgn(l[a].cross(l[b], p)) < r;
}
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

```
"Point.h" 39 lines
#define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
    int n = sz(poly), lo = 0, hi = n;
    if (extr(0)) return 0;
    while (lo + 1 < hi) {
        int m = (lo + hi) / 2;
        if (extr(m)) return m;
        int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
        (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
    }
    return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
    int endA = extrVertex(poly, (a - b).perp());
    int endB = extrVertex(poly, (b - a).perp());
    if (cmpL(endA) < 0 || cmpL(endB) > 0)
        return {-1, -1};
    array<int, 2> res;
    rep(i, 0, 2) {
        int lo = endB, hi = endA, n = sz(poly);
        while ((lo + 1) % n != hi) {
            int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
            (cmpL(m) == cmpL(endB) ? lo : hi) = m;
        }
        res[i] = (lo + !cmpL(hi)) % n;
        swap(endA, endB);
    }
    if (res[0] == res[1]) return {res[0], -1};
    if (!cmpL(res[0]) && !cmpL(res[1]))
        switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
            case 0: return {res[0], res[0]};
            case 2: return {res[1], res[1]};
        }
    return res;
}
```

6.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

```
"Point.h" 17 lines
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
    assert(sz(v) > 1);
    set<P> S;
    sort(all(v), [](P a, P b) { return a.y < b.y; });
    pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
    int j = 0;
    for (P p : v) {
        P d(1 + (ll)sqrt(ret.first), 0);
        while (v[j].y <= p.y - d.x) S.erase(v[j++]);
        auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
        for (; lo != hi; ++lo)
            ret = min(ret, {(p - *lo).dist2(), {p, *lo}});
        S.insert(p);
    }
    return ret.second;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

```
"Point.h" 63 lines
typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

bool on_x(const P& a, const P& b) { return a.x < b.x; }
bool on_y(const P& a, const P& b) { return a.y < b.y; }

struct Node {
    P pt; // if this is a leaf, the single point in it
    T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
    Node *first = 0, *second = 0;

    T distance(const P& p) { // min squared distance to a point
        T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
        T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
        return (P(x, y) - p).dist2();
    }

    Node(vector<P>&& vp) : pt(vp[0]) {
        for (P p : vp) {
            x0 = min(x0, p.x); x1 = max(x1, p.x);
            y0 = min(y0, p.y); y1 = max(y1, p.y);
        }
        if (vp.size() > 1) {
            // split on x if width >= height (not ideal...)
            sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
            // divide by taking half the array for each child (not
            // best performance with many duplicates in the middle)
            int half = sz(vp)/2;
            first = new Node({vp.begin(), vp.begin() + half});
            second = new Node({vp.begin() + half, vp.end()});
        }
    }
};

struct KDTree {
    Node* root;
    KDTree(const vector<P>& vp) : root(new Node({all(vp)})) {}

    pair<T, P> search(Node *node, const P& p) {
        if (!node->first) {
            // uncomment if we should not find the point itself:
```

```
        // if (p == node->pt) return {INF, P()};
        return make_pair((p - node->pt).dist2(), node->pt);
    }

    Node *f = node->first, *s = node->second;
    T bfirst = f->distance(p), bsec = s->distance(p);
    if (bfirst > bsec) swap(bsec, bfirst), swap(f, s);

    // search closest side first, other side if needed
    auto best = search(f, p);
    if (bsec < best.first)
        best = min(best, search(s, p));
    return best;
}

// find nearest point to a point, and its squared distance
// (requires an arbitrary operator< for Point)
pair<T, P> nearest(const P& p) {
    return search(root, p);
}
};
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order $\{t[0][0], t[0][1], t[0][2], t[1][0], \dots\}$, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

```
"Point.h" 88 lines
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t ll1; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point

struct Quad {
    Q rot, o; P p = arb; bool mark;
    P& F() { return r()->p; }
    Q& r() { return rot->rot; }
    Q prev() { return rot->o->rot; }
    Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
    ll1 p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
}

Q makeEdge(P orig, P dest) {
    Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
    H = r->o; r->r()->r() = r;
    rep(i, 0, 4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
    r->p = orig; r->F() = dest;
    return r;
}

void splice(Q a, Q b) {
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}

Q connect(Q a, Q b) {
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q, Q> rec(const vector<P>& s) {
    if (sz(s) <= 3) {
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if (sz(s) == 2) return {a, a->r()};
```

```

        E(b,c).rem(f.a);
        swap(FS[j--], FS.back());
        FS.pop_back();
    }
}

int nw = sz(FS);
rep(j,0,nw) {
    F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
    C(a, b, c); C(a, c, b); C(b, c, a);
}

for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
    A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
return FS;
};

```

8 lines

2 lines

6 lines

49 lines

```

struct Dinic {
    struct Edge {
        int to, rev;
        ll c, oc;
        ll flow() { return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>> adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
    void addEdge(int a, int b, ll c, ll rcap = 0) {
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f) {
        if (v == t || !f) return f;
        for (int& i = ptr[v]; i < sz(adj[v]); i++) {
            Edge& e = adj[v][i];
            if (lvl[e.to] == lvl[v] + 1)
                if ((ll p = dfs(e.to, t, min(f, e.c))) {
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
    }
}

```

```
        return 0;
    }
    ll calc(int s, int t) {
        ll flow = 0; q[0] = s;
        rep(L,0,31) do { // 'int L=30' maybe faster for random data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while (qi < qe && !lvl[t]) {
                int v = q[qi++];
                for (Edge e : adj[v])
                    if (!lvl[e.to] && e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
            while (lvl p = dfs(s, t, LLONG_MAX)) flow += p;
        } while (lvl[t]);
        return flow;
    }
    bool leftOfMinCut(int a) { return lvl[a] != 0; }
```

dinicMaxFlow.hh

Description: Min-cost max-flow. If costs can be negative, call setpi before maxflow, but note that negative cost cycles are not supported. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(FE \log(V))$ where F is max flow. $\mathcal{O}(VE)$ for setpi.

77 lines

```
const ll INF = numeric_limits<ll>::max() / 4;

struct MCMF {
    struct edge {
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge*>> ed;
    vi seen;
    vector<ll> dist, pi;
    vector<edge*> par;

    MCMF(int N) : N(N), ed(N), seen(N), dist(N), pi(N), par(N) {}

    void addEdge(int from, int to, ll cap, ll cost) {
        if (from == to) return;
        ed[from].push_back(edge{ from,to,sz(ed[to]),cap,cost,0 });
        ed[to].push_back(edge{ to,from,sz(ed[from])-1,0,-cost,0 });
    }

    void path(int s) {
        fill(all(seen), 0);
        fill(all(dist), INF);
        dist[s] = 0; ll di;

        __gnu_pbds::priority_queue<pair<ll, int>> q;
        vector<decltype(q)::point_iterator> its(N);
        q.push({ 0, s });

        while (!q.empty()) {
            s = q.top().second; q.pop();
            seen[s] = 1; di = dist[s] + pi[s];
            for (edge& e : ed[s]) if (!seen[e.to]) {
                ll val = di - pi[e.to] + e.cost;
                if (e.cap - e.flow > 0 && val < dist[e.to]) {
                    dist[e.to] = val;
                    par[e.to] = &e;
                    if (its[e.to] == q.end())
                        its[e.to] = q.push({ -dist[e.to], e.to });
                    else
                        q.modify(its[e.to], { -dist[e.to], e.to });
                }
            }
        }
    }
};
```

```
    }
    rep(i,0,N) pi[i] = min(pi[i] + dist[i], INF);
}

pair<ll, ll> maxflow(int s, int t) {
    ll totflow = 0, totcost = 0;
    while (path(s), seen[t]) {
        ll fl = INF;
        for (edge* x = par[t]; x; x = par[x->from])
            fl = min(fl, x->cap - x->flow);

        totflow += fl;
        for (edge* x = par[t]; x; x = par[x->from]) {
            x->flow += fl;
            ed[x->to][x->rev].flow -= fl;
        }
    }
    rep(i,0,N) for(edge& e : ed[i]) totcost += e.cost * e.flow;
    return {totflow, totcost/2};
}

// If some costs can be negative, call this before maxflow:
void setpi(int s) { // (otherwise, leave this out)
    fill(all(pi), INF); pi[s] = 0;
    int it = N, ch = 1; ll v;
    while (ch-- && it--)
        rep(i,0,N) if (pi[i] != INF)
            for (edge& e : ed[i]) if (e.cap)
                if ((v = pi[i] + e.cost) < pi[e.to])
                    pi[e.to] = v, ch = 1;
    assert(it >= 0); // negative cost cycle
}

};
```

minCut.hh

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

7.2 DFS Algorithms

topsort.hh

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.
Time: $\mathcal{O}(|V| + |E|)$

8 lines

```
vi topoSort(const vector<vi>& gr) {
    vi indeg(sz(gr)), q;
    for (auto& li : gr) for (int x : li) indeg[x]++;
    rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(i);
    rep(j,0,sz(q)) for (int x : gr[q[j]])
        if (--indeg[x] == 0) q.push_back(x);
    return q;
}
```

bridgesAndCutpoints.hh

Description: mostiki i tochki sochnleniya=
Time: $\mathcal{O}(N + E)$

45 lines

```
vector<int> g[MAXN];
bool used[MAXN];
int h[MAXN], d[MAXN];
set<int> ans;
void dfs_cutpoints(int v, int p = -1) {
    used[v] = 1;
    d[v] = h[v];
```

```
    int childrens = 0;
    for (int u : g[v]) {
        if (u == p) continue;
        if (!used[u]) {
            ++childrens;
            h[u] = h[v] + 1;
            dfs(u, v);
            d[v] = min(d[v], d[u]);
            if (h[v] <= d[u] && p != -1) {
                ans.insert(v);
            }
        } else {
            d[v] = min(d[v], h[u]);
        }
    }
    if (p == -1 && childrens > 1) {
        ans.insert(v);
    }
}

void dfs_bridges(int v, int p) {
    used[v] = 1;
    d[v] = h[v];
    for (int u : g[v]) {
        if (u == p) continue;
        if (!used[u]) {
            h[u] = h[v] + 1;
            dfs(u, v);
            d[v] = min(d[v], d[u]);
            if (h[v] < d[u]) {
                ans.push_back({v, u});
            }
        } else {
            d[v] = min(d[v], h[u]);
        }
    }
}
```

sccFind.hh

Description: SCC=
Time: $\mathcal{O}(N + E)$

42 lines

```
vector<vector<int>> g, gr;
vector<char> used;
vector<int> order, component;
void dfs1(int v) {
    used[v] = true;
    for (int u : g[v])
        if (!used[u])
            dfs1(u);
    order.push_back(v);
}

void dfs2(int v) {
    used[v] = true;
    component.push_back(v);
    for (int u : gr[v])
        if (!used[u])
            dfs2(u);
}

vector<vector<int>> SCC_find() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; ++i) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        gr[b].push_back(a);
    }
}
```

```
used.assign(n, false);
for (int i = 0; i < n; ++i)
    if (!used[i])
        dfs1(i);
used.assign(n, false);
vector <vector <int>> components;
for (int i = 0; i < n; ++i) {
    int v = order[n - 1 - i];
    if (!used[v]) {
        dfs2 (v);
        components.push_back(component);
        component.clear();
    }
}
return components;
}
```

2sat.hh

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars
Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

56 lines

```
struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n) {}

    int addVar() { // (optional)
        gr.emplace_back();
        gr.emplace_back();
        return N++;
    }

    void either(int f, int j) {
        f = max(2*f, -1-2*f);
        j = max(2*j, -1-2*j);
        gr[f].push_back(j^1);
        gr[j].push_back(f^1);
    }

    void setValue(int x) { either(x, x); }

    void atMostOne(const vi& li) { // (optional)
        if (sz(li) <= 1) return;
        int cur = ~li[0];
        rep(i,2,sz(li)) {
            int next = addVar();
            either(cur, ~li[i]);
            either(cur, next);
            either(~li[i], next);
            cur = ~next;
        }
        either(cur, ~li[1]);
    }
}
```

```
vi val, comp, z; int time = 0;
int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
```

```
low = min(low, val[e] ?: dfs(e));
if (low == val[i]) do {
    x = z.back(); z.pop_back();
    comp[x] = low;
    if (values[x>>1] == -1)
        values[x>>1] = x&1;
} while (x != i);
return val[i] = low;
}

bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
    return 1;
}
};
```

7.3 Smth

dijkstra.hh
Description: Bro= $\Rightarrow$ pora uzhe zabotat’))

37 lines

```
vector <pair <int, int>> g[MAXN];
int n;
vector<int> dijkstra_fast(int s) {
    vector<int> dist(n, INT_MAX);
    dist[s] = 0;
    set <pair <int, int>> q;
    q.emplace(0, s);
    while (!q.empty()) {
        auto [d, u] = *q.begin();
        q.erase(q.begin());
        for (auto [v, w] : g[u]) {
            if (dist[u] + w < dist[v]) {
                q.erase({dist[v], v});
                dist[v] = dist[u] + w;
                q.insert({dist[v], v});
            }
        }
    }
    return dist;
}

vector<int> dijkstra_slow(int s) {
    vector<int> dist(n, INT_MAX);
    vector<bool> visited(n, false);
    dist[s] = 0;
    for (int i = 0, u = -1; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (!visited[j] && (u == -1 || dist[j] < dist[u]))
                u = j;
        }
        if (dist[u] == INT_MAX) break;
        visited[u] = true;
        for (auto [v, w] : g[u])
            dist[v] = min(dist[v], dist[u] + w);
    }
    return dist;
}
```

7.4 Matching

dfsMatching.hh
Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and $btoa$ should be a vector full of -1’s of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it’s not matched.

Time: $\mathcal{O}(VE)$ 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
    if (btoa[j] == -1) return 1;
    vis[j] = 1; int di = btoa[j];
    for (int e : g[di])
        if (!vis[e] && find(e, g, btoa, vis)) {
            btoa[e] = di;
            return 1;
        }
    return 0;
}

int dfsMatching(vector<vi>& g, vi& btoa) {
    vi vis;
    rep(i,0,sz(g)) {
        vis.assign(sz(btoa), 0);
        for (int j : g[i])
            if (find(j, g, btoa, vis)) {
                btoa[j] = i;
                break;
            }
    }
    return sz(btoa) - (int)count(all(btoa), -1);
}
```

hungarian.hh

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes $cost[N][M]$, where $cost[i][j]$ = cost for $L[i]$ to be matched with $R[j]$ and returns (min cost, match), where $L[i]$ is matched with $R[match[i]]$. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}(N^2M)$

31 lines

```
pair<int, vi> hungarian(const vector<vi> &a) {
    if (a.empty()) return {0, {}};
    int n = sz(a) + 1, m = sz(a[0]) + 1;
    vi u(n), v(m), p(m), ans(n - 1);
    rep(i,1,n) {
        p[0] = i;
        int j0 = 0; // add "dummy" worker 0
        vi dist(m, INT_MAX), pre(m, -1);
        vector<bool> done(m + 1);
        do { // dijkstra
            done[j0] = true;
            int i0 = p[j0], j1, delta = INT_MAX;
            rep(j,1,m) if (!done[j]) {
                auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
                if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
                if (dist[j] < delta) delta = dist[j], j1 = j;
            }
            rep(j,0,m) {
                if (done[j]) u[p[j]] += delta, v[j] -= delta;
                else dist[j] -= delta;
            }
            j0 = j1;
        } while (p[j0]);
        while (j0) { // update alternating path
            int j1 = pre[j0];
            p[j0] = p[j1], j0 = j1;
        }
    }
    rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
    return {-v[0], ans}; // min cost
}
```

minVertexCover.hh

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"dfsMatching.hh"20 lines

```
vi cover(vector<vi>& g, int n, int m) {
    vi match(m, -1);
    int res = dfsMatching(g, match);
    vector<bool> lfound(n, true), seen(m);
    for (int it : match) if (it != -1) lfound[it] = false;
    vi q, cover;
    rep(i,0,n) if (lfound[i]) q.push_back(i);
    while (!q.empty()) {
        int i = q.back(); q.pop_back();
        lfound[i] = 1;
        for (int e : g[i]) if (!seen[e] && match[e] != -1) {
            seen[e] = true;
            q.push_back(match[e]);
        }
    }
    rep(i,0,n) if (!lfound[i]) cover.push_back(i);
    rep(i,0,m) if (seen[i]) cover.push_back(n+i);
    assert(sz(cover) == res);
    return cover;
}
```

7.5 Coloring

edgeColoring.hh

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

"Time: $\mathcal{O}(NM)$ "31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for (pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for (pii e : eds) {
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while (d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        while (adj[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], e = cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
        for (int y : {fan[0], u, end})
            for (int& z = free[y] = 0; adj[y][z] != -1; z++);
    }
    rep(i,0,sz(eds))
        for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
    return ret;
}
```

7.6 Heuristics

maxCliquesSet.hh

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

"Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs"12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
    if (!P.any()) { if (!X.any()) f(R); return; }
    auto q = (P | X)._Find_first();
    auto cands = P & ~eds[q];
    rep(i,0,sz(eds)) if (cands[i]) {
        R[i] = 1;
        cliques(eds, f, P & eds[i], X & eds[i], R);
        R[i] = P[i] = 0; X[i] = 1;
    }
}
```

maxClique.hh

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

"49 lines"

```
typedef vector<bitset<200>> vb;
struct Maxclique {
    double limit=0.025, pk=0;
    struct Vertex { int i, d=0; };
    typedef vector<Vertex> vv;
    vb e;
    vv V;
    vector<vi> C;
    vi qmax, q, S, old;
    void init(vv& r) {
        for (auto& v : r) v.d = 0;
        for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
        sort(all(r), [](auto a, auto b) { return a.d > b.d; });
        int mxD = r[0].d;
        rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
    }
    void expand(vv& R, int lev = 1) {
        S[lev] += S[lev - 1] - old[lev];
        old[lev] = S[lev - 1];
        while (sz(R)) {
            if (sz(q) + R.back().d <= sz(qmax)) return;
            q.push_back(R.back().i);
            vv T;
            for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
            if (sz(T)) {
                if (S[lev]++ / ++pk < limit) init(T);
                int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
                C[1].clear(), C[2].clear();
                for (auto v : T) {
                    int k = 1;
                    auto f = [&](int i) { return e[v.i][i]; };
                    while (any_of(all(C[k]), f)) k++;
                    if (k > mxk) mxk = k, C[mxk + 1].clear();
                    if (k < mnk) T[j++] .i = v.i;
                    C[k].push_back(v.i);
                }
                if (j > 0) T[j - 1].d = 0;
                rep(k,mnk,mxk + 1) for (int i : C[k])
                    T[j].i = i, T[j++].d = k;
                expand(T, lev + 1);
            } else if (sz(q) > sz(qmax)) qmax = q;
            q.pop_back(), R.pop_back();
        }
    }
}
```

```
    }
}
vi maxClique() { init(V), expand(V); return qmax; }
Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
}
};
```

7.7 Trees

HLD.hh

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most $\log(n)$ light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.

"Time: $\mathcal{O}\left((\log N)^2\right)$ "46 lines

"../DataStructures/lazySegmentTree.hh"

```
template <bool VALS_EDGES> struct HLD {
    int N, tim = 0;
    vector<vi> adj;
    vi par, siz, rt, pos;
    Node *tree;
    HLD(vector<vi> adj_)
        : N(sz(adj_)), adj(adj_), par(N, -1), siz(N, 1),
          rt(N),pos(N),tree(new Node(0, N)){ dfsSz(0); dfsHld(0); }
    void dfsSz(int v) {
        if (par[v] != -1) adj[v].erase(find(all(adj[v]), par[v]));
        for (int& u : adj[v]) {
            par[u] = v;
            dfsSz(u);
            siz[v] += siz[u];
            if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
        }
    }
    void dfsHld(int v) {
        pos[v] = tim++;
        for (int u : adj[v]) {
            rt[u] = (u == adj[v][0] ? rt[v] : u);
            dfsHld(u);
        }
    }
    template <class B> void process(int u, int v, B op) {
        for (; rt[u] != rt[v]; v = par[rt[v]]) {
            if (pos[rt[u]] > pos[rt[v]]) swap(u, v);
            op(pos[rt[v]], pos[v] + 1);
        }
        if (pos[u] > pos[v]) swap(u, v);
        op(pos[u] + VALS_EDGES, pos[v] + 1);
    }
    void modifyPath(int u, int v, int val) {
        process(u, v, [&](int l, int r) { tree->add(l, r, val); });
    }
    int queryPath(int u, int v) { // Modify depending on problem
        int res = -1e9;
        process(u, v, [&](int l, int r) {
            res = max(res, tree->query(l, r));
        });
        return res;
    }
    int querySubtree(int v) { // modifySubtree is similar
        return tree->query(pos[v] + VALS_EDGES, pos[v] + siz[v]);
    }
};
```

directedMST.hh

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E \log V)$

"../data-structures/persistentDSU.hh"60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;
    void prop() {
        key.w += delta;
        if (l) l->delta += delta;
        if (r) r->delta += delta;
        delta = 0;
    }
    Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
    if (!a || !b) return a ? b;
    a->prop(), b->prop();
    if (a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
    RollbackUF uf(n);
    vector<Node*> heap(n);
    for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
    ll res = 0;
    vi seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n, in(n, {-1,-1})), comp;
    deque<tuple<int, int, vector<Edge>>> cyscs;
    rep(s,0,n) {
        int u = s, qi = 0, w;
        while (seen[u] < 0) {
            if (!heap[u]) return {-1,{};};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if (seen[u] == s) {
                Node* cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while (uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cyscs.push_front({u, time, {&Q[qi], &Q[end]}});
            }
        }
        rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
    }

    for (auto& [u,t,comp] : cyscs) { // restore sol (optional)
        uf.rollback(t);
        Edge inEdge = in[u];
        for (auto& e : comp) in[uf.find(e.b)] = e;
        in[uf.find(inEdge.b)] = inEdge;
    }
    rep(i,0,n) par[i] = in[i].a;
    return {res, par};
}
```

7.8 Math

7.8.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $\text{mat}[a][b]--$, $\text{mat}[b][b]++$ (and $\text{mat}[b][a]--$, $\text{mat}[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Numerical (8)

8.1 Polynomials and recurrences

Polynomial.h17 lines

```
struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for (int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
    void diff() {
        rep(i,1,sz(a)) a[i-1] = i*a[i];
        a.pop_back();
    }
    void divroot(double x0) {
        double b = a.back(), c; a.back() = 0;
        for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
        a.pop_back();
    }
};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.
Usage: polyRoots({{2,-3,1}},-1e9,1e9) // solve $x^2-3x+2 = 0$
Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

"Polynomial.h"23 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
    if (sz(p.a) == 2) { return {p.a[0]/p.a[1]}; }
    vector<double> ret;
    Poly der = p;
    der.diff();
    auto dr = polyRoots(der, xmin, xmax);
    dr.push_back(xmin-1);
    dr.push_back(xmax+1);
    sort(all(dr));
    rep(i,0,sz(dr)-1) {
        double l = dr[i], h = dr[i+1];
        bool sign = p(l) > 0;
        if (sign ^ (p(h) > 0)) {
            rep(it,0,60) { // while (h - l > 1e-8)
                double m = (l + h) / 2, f = p(m);
                if ((f <= 0) ^ sign) l = m;
```

```
            else h = m;
        }
        ret.push_back((l + h) / 2);
    }
}
return ret;
}
```

PolyInterpolate.h

Description: Given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi)$, $k = 0 \dots n-1$.
Time: $\mathcal{O}(n^2)$

13 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
    vd res(n), temp(n);
    rep(k,0,n-1) rep(i,k+1,n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k,0,n) rep(i,0,n) {
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

BerlekampMassey.h

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}
Time: $\mathcal{O}(N^2)$

20 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i,0,n) { ++m;
        ll d = s[i] % mod;
        rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C; ll coef = d * modpow(b, mod-2) % mod;
        rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
    for (ll& x : C) x = (mod - x) % mod;
    return C;
}
```

LinearRecurrence.h

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: linearRec({0, 1}, {1, 1}, k) // k 'th Fibonacci number
Time: $\mathcal{O}(n^2 \log k)$

26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b) {
```

```

Poly res(n * 2 + 1);
rep(i,0,n+1) rep(j,0,n+1)
    res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
for (int i = 2 * n; i > n; --i) rep(j,0,n)
    res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
res.resize(n + 1);
return res;
};

Poly pol(n + 1), e(pol);
pol[0] = e[1] = 1;

for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
}

ll res = 0;
rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
return res;
}
```

8.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is ϵ *ps*. Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.

Usage: double func(double x) { return 4+x+.3*x*x; }
double xmin = gss(-1000,1000,func);

Time: $\mathcal{O}(\log((b - a)/\epsilon))$

```
double gss(double a, double b, double (*f)(double)) {
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while (b-a > eps)
        if (f1 < f2) { //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        } else {
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}
```

HillClimbing.h

Description: Poor man’s optimization for unimodal functions.

```
typedef array<double, 2> P;

template<class F> pair<double, P> hillClimb(P start, F f) {
    pair<double, P> cur(f(start), start);
    for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
        rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
            P p = cur.second;
            p[0] += dx*jmp;
            p[1] += dy*jmp;
            cur = min(cur, make_pair(f(p), p));
        }
    }
    return cur;
}
```

Integrate.h

Description: Simple integration of a function over an interval using Simpson’s rule. The error should be proportional to h^4 , although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
    double h = (b - a) / 2 / n, v = f(a) + f(b);
    rep(i,1,n*2)
        v += f(a + i*h) * (i&1 ? 4 : 2);
    return v * h / 3;
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson’s rule.

Usage: double sphereVolume = quad(-1, 1, [](double x) {
return quad(-1, 1, [&](double y) {
return quad(-1, 1, [&](double z) {
return x*x + y*y + z*z < 1; } } } });

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
    d c = (a + b) / 2;
    d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
    if (abs(T - S) <= 15 * eps || b - a < 1e-10)
        return T + (T - S) / 15;
    return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
}

template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
    return rec(f, a, b, eps, S(a, b));
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b, x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.

Usage: vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vvd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);

Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

```
typedef double T; // long double, Rational, double + modP>...
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j

struct LPSolver {
    int m, n;
    vi N, B;
    vvd D;

    LPSolver(const vvd& A, const vd& b, const vd& c) :
        m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
        rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
        rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i]; }
        rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
        N[n] = -1; D[m+1][n] = 1;
    }
```

```

}

void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
        T *b = D[i].data(), inv2 = b[s] * inv;
        rep(j,0,n+2) b[j] -= a[j] * inv2;
        b[s] = a[s] * inv2;
    }
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
}

bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
        int s = -1;
        rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
        if (D[x][s] >= -eps) return true;
        int r = -1;
        rep(i,0,m) {
            if (D[i][s] <= eps) continue;
            if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                < MP(D[r][n+1] / D[r][s], B[r])) r = i;
        }
        if (r == -1) return false;
        pivot(r, s);
    }
}

T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
        pivot(r, n);
        if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
        rep(i,0,m) if (B[i] == -1) {
            int s = 0;
            rep(j,1,n+1) ltj(D[i]);
            pivot(i, s);
        }
    }

    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
}
};
```

8.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.

Time: $\mathcal{O}(N^3)$

```
double det(vector<vector<double>>& a) {
    int n = sz(a); double res = 1;
    rep(i,0,n) {
        int b = i;
        rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if (i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if (res == 0) return 0;
        rep(j,i+1,n) {
            double v = a[j][i] / a[i][i];
            if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```



```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
    int n = sz(b); vi tr(n);
    rep(i,0,n-1) {
        if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[++i] = 1;
        } else {
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
}
for (int i = n; i--;) {
    if (tr[i]) {
        swap(b[i], b[i-1]);
        diag[i-1] = diag[i];
        b[i] /= super[i-1];
    } else {
```

8.4 Fourier transforms

FastFourierTransform.h

Description: $\text{fft}(a)$ computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

```
template<class db>
struct Complex {
    db real, image;

    Complex(db _real = 0, db _image = 0) : real(_real), image(_image) {}

    Complex operator+(const Complex &other) { return Complex(
        real + other.real, image + other.image); }

    Complex operator-(const Complex &other) { return Complex(
        real - other.real, image - other.image); }

    Complex operator*(const Complex &other) {
        return Complex(real * other.real - image * other.image,
            real * other.image + image * other.real);
    }

    Complex operator/(db x) { return Complex(real / x, image /
        x); }
};

template<class db>
class FFT {
    //double vs long double!!!
public:
    const db PI = 4 * atan2(1, 1);

    FFT(int _n) : size(1 << _n), n(_n) {
        revers.resize(size), root.resize(size);
        fftA.resize(size), fftB.resize(size);
        for (int i = 0; i < size / 2; i++) {
            root[i] = Complex<db>(cos(2 * PI * i / size), sin(2
                * PI * i / size));
            root[i + size / 2] = Complex<db>(-root[i].real, -
                root[i].image);
        }
    }

    void fft(vector<Complex<db>> &poly, int newN) {
        revers[0] = 0;
        for (int i = 1; i < (1 << newN); i++) {
            if (i % 2 == 0) revers[i] = revers[i / 2] / 2;
            else revers[i] = revers[i / 2] / 2 + (1 << (newN -
                1));
            if (revers[i] < i) swap(poly[revers[i]], poly[i]);
        }

        for (int level = 1; level <= newN; level++)
            for (int block = 0; block < (1 << (newN - level));
                block++)
                for (int step = 0; step < (1 << (level - 1));
                    step++) {
                    int num1 = (block << level) + step;
                    int num2 = num1 + (1 << (level - 1));
                    int valA = poly[num1];
                    Complex<db> valB = poly[num2];
```

FastFourierTransform NumberTheoreticTransform

```
Complex<db> valB = root[step << (n - level)
    ] * poly[num2];
poly[num1] = valA + valB;
poly[num2] = valA - valB;
    }
}

void rev_fft(vector<Complex<db>> &poly, int step) {
    fft(poly, step);
    reverse(poly.begin() + 1, poly.begin() + (1 << step));
    for (int i = 0; i < (1 << step); i++) poly[i] = poly[i]
        / (1 << step);
}

vector<db> prod(const vector<db> &A, const vector<db> &B,
    int step) {
    fill(fftA.begin(), fftA.begin() + (1 << step), 0);
    fill(fftB.begin(), fftB.begin() + (1 << step), 0);
    for (int i = 0; i < (int) A.size(); i++) fftA[i] = A[i]
        ;
    for (int i = 0; i < (int) B.size(); i++) fftB[i] = B[i]
        ;

    fft(fftA, step);
    fft(fftB, step);
    for (int i = 0; i < (1 << step); i++) fftA[i] = fftA[i]
        * fftB[i];
    rev_fft(fftA, step);

    vector<db> result(1 << step);
    for (int i = 0; i < (1 << step); i++) result[i] = fftA[
        i].real;
    return result;
}

private:
    int size, n;
    vector<Complex<db>> root;
    vector<int> revers;

    vector<Complex<db>> fftA, fftB;
};
```

NumberTheoreticTransform.h

Description: $\text{ntt}(a)$ computes $\hat{f}(k) = \sum_x a[x]g^{xk}$ for all k , where $g = \text{root}^{(mod-1)/N}$.

Time: $\mathcal{O}(N \log N)$

```
namespace NT {
    int prod(int a, int b, int m) {
        return (ul)a * b % m;
    }

    int inv(int a, int m) {
        if (a == 1) return 1;
        return (1 - ll(inv(m % a, a)) * m) / a + m;
    }

    int sum(int a, int b, int m) {
        a += b; if (a >= m || a < b) a -= m;
        return a;
    }

    int dif(int a, int b, int m) {
        b = a - b; if (b < 0 || b > a) b += m;
        return b;
    }

    ul sumul(ul a, ul b, ul m) {
        a += b; if (a >= m || a < b) a -= m;
        return a;
    }

    ul diful(ul a, ul b, ul m) {

```

```
if ((b = a - b) > a) b += m;
return b;
}

int powmod(int a, ul b, int m) {
    int ans = 1;
    while (b) {
        if (b & 1) ans = prod(ans, a, m);
        if (b >= 1) a = prod(a, a, m);
    }
    return ans;
}

class NTT {
public:
    const int n; // <= 23
    const int size; // == 1 << n
    const int MOD = 998244353, G = 3;
    const int ROOT; // = 565042129 for n == 20;
    const int imaginary_unit = 911660635; // i^2 = MOD - 1

    NTT(int n = 20) : n(n), size(1 << n), ROOT(NT::powmod(G
        , (MOD - 1) >> n, MOD)), revers(size), root(size,
        1),
        fftA(size), fftB(size) {
        for (int i = 1; i < size; i++)
            root[i] = (ll) root[i - 1] * ROOT % MOD;
    }

    void fft(vector<int> &poly, int newN) {
        revers[0] = 0;
        for (int i = 1; i < (1 << newN); i++) {
            if (i % 2 == 0) revers[i] = revers[i / 2] / 2;
            else revers[i] = revers[i / 2] / 2 + (1 << (
                newN - 1));
            if (revers[i] < i) swap(poly[revers[i]], poly[i]
                );
        }

        for (int level = 1; level <= newN; level++)
            for (int block = 0; block < (1 << (newN - level
                )); block++)
                for (int step = 0; step < (1 << (level - 1)
                    ); step++) {
                    int num1 = (block << level) + step;
                    int num2 = num1 + (1 << (level - 1));
                    int valA = poly[num1];
                    int valB = (ll) root[step << (n - level
                        )] * poly[num2] % MOD;
                    poly[num1] = (valA + valB) % MOD;
                    poly[num2] = (valA - valB + MOD) % MOD;
                }
    }

    void rev_fft(vector<int> &poly, int step) {
        fft(poly, step);
        reverse(poly.begin() + 1, poly.begin() + (1 << step
            ));
        int inv_size = NT::powmod((1 << step), MOD - 2, MOD
            );
        for (int i = 0; i < (1 << step); i++) poly[i] = (ll
            ) poly[i] * inv_size % MOD;
    }

    template<class Iter>
    vector<int> multiply(Iter Ab, Iter Ae, Iter Bb, Iter Be
        , int step) {
        if (Ab == Ae || Bb == Be) return {};
        fill(fftA.begin(), fftA.begin() + (1 << step), 0);

```

```

fill(fftB.begin(), fftB.begin() + (1 << step), 0);
for (auto it = Ab; it != Ae; ++it) fftA[it - Ab] =
    *it % MOD;
for (auto it = Bb; it != Be; ++it) fftB[it - Bb] =
    *it % MOD;

fft(fftA, step);
fft(fftB, step);
for (int i = 0; i < (1 << step); i++) fftA[i] = (1l
    ) fftA[i] * fftB[i] % MOD;
rev_fft(fftA, step);

vector<int> result(1 << step);
for (int i = 0; i < (1 << step); i++) result[i] =
    fftA[i];
return result;
}

vector<int> multiply(const vector<int> &A, const vector
    <int> &B, int step) {
    return multiply(A.begin(), A.end(), B.begin(), B.
        end(), step);
}
// returns a vector of size exactly n
vi prod(const vi &A, const vi &B, const int n) {
    int as = min<int>(A.size(), n), bs = min<int>(B.
        size(), n);
    vi ans = multiply(A.begin(), A.begin() + as, B.
        begin(), B.begin() + bs, 32 - __builtin_clz(as
            + bs - 2));
    ans.resize(n);
    return ans;
}
// returns a vector of size A.size() + B.size() - 1 or
    0
vi prod(const vi &A, const vi &B) {
    if (A.empty() || B.empty()) return {};
    return prod(A, B, A.size() + B.size() - 1);
}

vi sum(const vi &A, const vi &B) const {
    vi ans(max(A.size(), B.size()));
    for (int i = 0; i < ans.size(); ++i)
        if (i >= A.size()) ans[i] = B[i];
        else if (i >= B.size()) ans[i] = A[i];
        else ans[i] = NT::sum(A[i], B[i], MOD);
    return ans;
}

vi dif(const vi &A, const vi &B) const {
    vi ans(max(A.size(), B.size()));
    for (int i = 0; i < ans.size(); ++i)
        if (i >= A.size()) ans[i] = NT::dif(0, B[i],
            MOD);
        else if (i >= B.size()) ans[i] = A[i];
        else ans[i] = NT::dif(A[i], B[i], MOD);
    return ans;
}
// A[0] != 0
vi inverse(const vi &A, const int n) {
    vi a(1, A[0]);
    vi b(1, NT::inv(A[0], MOD));
    for (int bits = 1; b.size() < n; ++bits) {
        copy(A.begin() + min<int>(A.size(), b.size()),
            A.begin() + min<int>(1 << bits, A.size()),
            back_inserter(a));
        b = prod(b, dif(vi(1, 2), prod(a, b, 1 << bits)
            ), min(n, 1 << bits));
    }
}

```

```

return b;
}

vi derivative(const vi &A) const {
    if (A.size() <= 1) return {};
    vi B(A.size() - 1);
    for (int i = 1; i < A.size(); ++i)
        B[i - 1] = NT::prod(A[i], i, MOD);
    return B;
}

vi factorials(const int n) const {
    vi fact(n, 1);
    for (int i = 2; i < n; ++i)
        fact[i] = NT::prod(i, fact[i - 1], MOD);
    return fact;
}

vi inverses(const int n) const {
    vi fact = factorials(n);
    int ifact = NT::inv(fact.back(), MOD);
    for (int i = n - 1; i >= 2; --i) {
        fact[i] = NT::prod(fact[i - 1], ifact, MOD);
        ifact = NT::prod(ifact, i, MOD);
    }
    return fact;
}
// ans[0] == 0
vi integral(const vi &A) const {
    vi B(A.size() + 1);
    vi invs = inverses(B.size());
    for (int i = 1; i <= A.size(); ++i)
        B[i] = NT::prod(A[i - 1], invs[i], MOD);
    return B;
}
// A[0] == 1, ans[0] == 0
vi logarithm(vi A, const int n) {
    A.resize(min<int>(A.size(), n)); if (n == 0) return
        A;
    A = prod(derivative(A), inverse(A, n - 1), n - 1);
    return integral(A);
}
// A[0] == 0, ans[0] == 1
vi exponent(const vi &A, const int n) {
    vi a(1);
    vi b(1, 1);
    for (int bits = 1; b.size() < n; ++bits) {
        vi ln_b = logarithm(b, min(1 << bits, n));
        copy(A.begin() + b.size(), A.begin() + min<int>
            >(1 << bits, A.size()), back_inserter(a));
        ln_b = dif(a, ln_b);
        ln_b[0] = NT::sum(ln_b[0], 1, MOD);
        b = prod(b, ln_b, min(1 << bits, n));
    }
    return b;
}
// A[0] == 1, ans[0] == 1, alpha is rational
vi power(vi A, const int alpha, const int n) {
    A = logarithm(A, n);
    for (int &a: A)
        a = NT::prod(a, alpha, MOD);
    A = exponent(A, n);
    return A;
}
// A[0] == 0, ans[0] == 0
vi sin(const vi &A, const int n) {
    static int MINUS_I_HALF = NT::prod(imaginary_unit,
        (MOD - 1) / 2, MOD);
    vi a = A; a.resize(min<int>(n, A.size()));
}

```

```

for (int &i: a) i = NT::prod(i, imaginary_unit, MOD
    );
vi exp_1 = exponent(a, n);
vi exp_2 = inverse(exp_1, n);
exp_1 = dif(exp_1, exp_2);
for (int &i : exp_1) i = NT::prod(i, MINUS_I_HALF,
    MOD);
return exp_1;
}
// A[0] == 0, ans[0] == 1
vi cos(const vi &A, const int n) {
    static int HALF = (MOD + 1) / 2;
    vi a = A; a.resize(min<int>(n, A.size()));
    for (int &i: a) i = NT::prod(i, imaginary_unit, MOD
        );
    vi exp_1 = exponent(a, n);
    vi exp_2 = inverse(exp_1, n);
    exp_1 = sum(exp_1, exp_2);
    for (int &i : exp_1) i = NT::prod(i, HALF, MOD);
    return exp_1;
}
// A[0] == 0, ans[0] == 0; tg(x) = 2i / (e^{2ix} + 1) -
    i
vi tg(vi A, const int n) {
    const int TWO_I = NT::sum(imaginary_unit,
        imaginary_unit, MOD);
    for (int &i : A) i = NT::prod(i, TWO_I, MOD);
    A = exponent(A, n); A[0] = NT::sum(A[0], 1, MOD); A
        = inverse(A, n);
    for (int &i : A) i = NT::prod(i, TWO_I, MOD);
    A[0] = NT::dif(A[0], imaginary_unit, MOD); return A
        ;
}
// A[0] == 0, ans[0] == 0
vi arcsin(const vi &A, const int n) {
    static int MINUS_HALF = (MOD - 1) / 2;
    return integral(prod(derivative(A), power(dif(vi(1,
        1), prod(A, A, n - 1))), MINUS_HALF, n - 1), n
        - 1));
}
// A[0] == 0, ans[0] == 0
vi arctg(const vi &A, const int n) {
    if (n <= 1) return vi(n);
    vi A2P1 = prod(A, A, n - 1); A2P1[0] = NT::sum(A2P1
        [0], 1, MOD);
    return integral(prod(derivative(A), inverse(A2P1, n
        - 1), n - 1));
}
// A[0] == 0, ans[0] == 0
vi sh(vi A, const int n) {
    static int HALF = (MOD + 1) / 2;
    A = exponent(A, n); A = dif(A, inverse(A, n));
    for (int &i: A) i = NT::prod(i, HALF, MOD);
    return A;
}
// A[0] == 0, ans[0] == 1
vi ch(vi A, const int n) {
    static int HALF = (MOD + 1) / 2;
    A = exponent(A, n); A = sum(A, inverse(A, n));
    for (int &i: A) i = NT::prod(i, HALF, MOD);
    return A;
}
// A[0] == 0, ans[0] == 0; 1 - 2 / (e^{2x} + 1)
vi th(vi A, const int n) {
    A = sum(A, A); A = exponent(A, n);
    A[0] = NT::sum(A[0], 1, MOD); A = inverse(A, n);
    A = sum(A, A); A = dif(vi(1, 1), A);
    return A;
}
}

```

```
// A[0] == 0, ans[0] == 0
vi arcsh(const vi &A, const int n) {
    static int MINUS_HALF = (MOD - 1) / 2;
    return integral(prod(derivative(A), power(sum(vi(1, 1), prod(A, A, n - 1)), MINUS_HALF, n - 1), n - 1));
}
// A[0] == 0, ans[0] == 0
vi arcth(const vi &A, const int n) {
    vi A2P1 = prod(A, A, n - 1); A2P1 = dif(vi(1, 1), A2P1);
    return integral(prod(derivative(A), inverse(A2P1, n - 1, n - 1)));
}
vi EGF(vi ogf) {
    int ifact = 1, n = ogf.size();
    for (int i = 2; i < n; ++i) ifact = NT::prod(ifact, i, MOD);
    ifact = NT::inv(ifact, MOD);
    for (int i = n - 1; i >= 2; --i) {
        ogf[i] = NT::prod(ogf[i], ifact, MOD);
        ifact = NT::prod(ifact, i, MOD);
    }
    return ogf;
}
vi OGF(vi egf) {
    int fact = 1, n = egf.size();
    for (int i = 2; i < n; ++i) {
        fact = NT::prod(fact, i, MOD);
        egf[i] = NT::prod(egf[i], fact, MOD);
    }
    return egf;
}
// A[0] == 0, B[0] == 0
vi euler_semitransform(const vi &A, const int n) {
    vi B(n);
    vi invs = inverses(n);
    for (int k = 1; k < n; ++k)
        for (int i = 1, j = k; j < n; ++i, j += k)
            B[j] = NT::sum(B[j], NT::prod(A[i], invs[k], MOD), MOD);
    return B;
}
// A[0] == 0, B[0] == 1
vi euler_transform(const vi &A, const int n) {
    return exponent(euler_semitransform(A, n), n);
}
// A[0] == 0, B[0] == 0
vi inverse_euler_semitransform(vi B, const int n) {
    vi fact(n, 1), invs(n, 1);
    for (int i = 2; i < n; ++i) fact[i] = NT::prod(i, fact[i - 1], MOD);
    int ifact = NT::inv(fact[n - 1], MOD);
    for (int i = n - 1; i >= 2; --i) {
        invs[i] = NT::prod(ifact, fact[i - 1], MOD);
        ifact = NT::prod(ifact, i, MOD);
    }
    for (int i = 1; i < n; ++i)
        for (int k = 2, j = 2 * i; j < n; ++k, j += i)
            B[j] = NT::dif(B[j], NT::prod(B[i], invs[k], MOD), MOD);
    return B;
}
// A[0] == 0, B[0] == 1
vi inverse_euler_transform(const vi &B, const int n) {
    return inverse_euler_semitransform(logarithm(B, n), n);
}
```

```
private:
    vector<int> root, revers, fftA, fftB;
};
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

```
void FST(vi& a, bool inv) {
    for (int n = sz(a), step = 1; step < n; step *= 2) {
        for (int i = 0; i < n; i += 2 * step) rep(j, i, i + step) {
            int &u = a[j], &v = a[j + step]; tie(u, v) =
                inv ? pii(v - u, u) : pii(v, u + v); // AND
                inv ? pii(v, u - v) : pii(u + v, u); // OR
                pii(u + v, u - v); // XOR
        }
        if (inv) for (int& x : a) x /= sz(a); // XOR only
    }
}
vi conv(vi a, vi b) {
    FST(a, 0); FST(b, 0);
    rep(i, 0, sz(a)) a[i] *= b[i];
    FST(a, 1); return a;
}
```

Strings (9)

KMP.h

Description: $\text{pi}[x]$ computes the length of the longest prefix of s that ends at x , other than $s[0..x]$ itself (abacaba -> 0010123). Can be used to find all occurrences of a string.

Time: $\mathcal{O}(n)$

```
vi pi(const string& s) {
    vi p(sz(s));
    rep(i, 1, sz(s)) {
        int g = p[i - 1];
        while (g && s[i] != s[g]) g = p[g - 1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
```

```
vi match(const string& s, const string& pat) {
    vi p = pi(pat + '\0' + s), res;
    rep(i, sz(p) - sz(s), sz(p))
        if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

Zfunc.h

Description: $z[i]$ computes the length of the longest common prefix of $s[i:]$ and s , except $z[0] = 0$. (abacaba -> 0010301)

Time: $\mathcal{O}(n)$

```
vi Z(const string& S) {
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i, 1, sz(S)) {
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
}
```

```
return z;
}
```

Manacher.h

Description: For each position in a string, computes $p[0][i]$ = half length of longest even palindrome around pos i , $p[1][i]$ = longest odd (half rounded down).

Time: $\mathcal{O}(N)$

```
array<vi, 2> manacher(const string& s) {
    int n = sz(s);
    array<vi, 2> p = {vi(n + 1), vi(n)};
    rep(z, 0, 2) for (int i = 0, l = 0, r = 0; i < n; i++) {
        int t = r - i + 1;
        if (i < r) p[z][i] = min(t, p[z][l + t]);
        int L = i - p[z][i], R = i + p[z][i] - 1;
        while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
            p[z][i]++, L--, R++;
        if (R > r) l = L, r = R;
    }
    return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.

Usage: `rotate(v.begin(), v.begin() + minRotation(v), v.end());`

Time: $\mathcal{O}(N)$

```
int minRotation(string s) {
    int a = 0, N = sz(s); s += s;
    rep(b, 0, N) rep(k, 0, N) {
        if (a + k == b || s[a + k] < s[b + k]) {b += max(0, k - 1); break;}
        if (s[a + k] > s[b + k]) {a = b; break;}
    }
    return a;
}
```

SuffixArray.h

Description: Builds suffix array for a string. $\text{sa}[i]$ is the starting index of the suffix which is i 'th in the sorted suffix array. The returned vector is of size $n + 1$, and $\text{sa}[0] = n$. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: $\text{lcp}[i] = \text{lcp}(\text{sa}[i], \text{sa}[i - 1])$, $\text{lcp}[0] = 0$. The input string must not contain any zero bytes.

Time: $\mathcal{O}(n \log n)$

```
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string& s, int lim = 256) { // or basic_string<int>
        int n = sz(s) + 1, k = 0, a, b;
        vi x(all(s)), y(n), ws(max(n, lim));
        x.push_back(0), sa = lcp = y, iota(all(sa), 0);
        for (int j = 0, p = 0; j < n; j = max(1, j * 2), lim = p) {
            p = j, iota(all(y), n - j);
            rep(i, 0, n) if (sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i, 0, n) ws[x[i]]++;
            rep(i, 1, lim) ws[i] += ws[i - 1];
            for (int i = n; i--;) sa[--ws[x[i]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i, 1, n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for (int i = 0, j; i < n - 1; lcp[x[i + 1]] = k)
            for (k && k--, j = sa[x[i] - 1];
                s[i + k] == s[j + k]; k++);
    }
};
```

SuffixTree.h

Description: Ukkonen’s algorithm for online suffix tree construction. Each node contains indices [l, r) into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r) substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).

Time: $\mathcal{O}(26N)$

```
50 lines
struct SuffixTree {
    enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
    int toi(char c) { return c - 'a'; }
    string a; // v = cur node, q = cur position
    int t[N][ALPHA], l[N], r[N], p[N], s[N], v=0, q=0, m=2;

    void ukkadd(int i, int c) { suff:
        if (r[v]<=q) {
            if (t[v][c]==-1) { t[v][c]=m; l[m]=i;
                p[m++]=v; v=s[v]; q=r[v]; goto suff; }
            v=t[v][c]; q=l[v];
        }
        if (q==-1 || c==toi(a[q])) q++; else {
            l[m+1]=i; p[m+1]=m; l[m]=l[v]; r[m]=q;
            p[m]=p[v]; t[m][c]=m+1; t[m][toi(a[q])]=v;
            l[v]=q; p[v]=m; t[p[m]][toi(a[l[m])]]=m;
            v=s[p[m]]; q=l[m];
            while (q<r[m]) { v=t[v][toi(a[q])]; q+=r[v]-l[v]; }
            if (q==r[m]) s[m]=v; else s[m]=m+2;
            q=r[v]-(q-r[m]); m+=2; goto suff;
        }
    }

    SuffixTree(string a) : a(a) {
        fill(r,r+N,sz(a));
        memset(s, 0, sizeof s);
        memset(t, -1, sizeof t);
        fill(t[1],t[1]+ALPHA,0);
        s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
        rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
    }

    // example: find longest common substring (uses ALPHA = 28)
    pii best;
    int lcs(int node, int i1, int i2, int olen) {
        if (l[node] <= i1 && i1 < r[node]) return 1;
        if (l[node] <= i2 && i2 < r[node]) return 2;
        int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
        rep(c,0,ALPHA) if (t[node][c] != -1)
            mask |= lcs(t[node][c], i1, i2, len);
        if (mask == 3)
            best = max(best, {len, r[node] - len});
        return mask;
    }
    static pii LCS(string s, string t) {
        SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
        st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
        return st.best;
    }
};
```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(–, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.

Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

```
66 lines
struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; // change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v) { memset(next, v, sizeof(next)); }
    };
    vector<Node> N;
    vi backp;
    void insert(string& s, int j) {
        assert(!s.empty());
        int n = 0;
        for (char c : s) {
            int& m = N[n].next[c - first];
            if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
            else n = m;
        }
        if (N[n].end == -1) N[n].start = j;
        backp.push_back(N[n].end);
        N[n].end = j;
        N[n].nmatches++;
    }
    AhoCorasick(vector<string>& pat) : N(1, -1) {
        rep(i,0,sz(pat)) insert(pat[i], i);
        N[0].back = sz(N);
        N.emplace_back(0);

        queue<int> q;
        for (q.push(0); !q.empty(); q.pop()) {
            int n = q.front(), prev = N[n].back;
            rep(i,0,alpha) {
                int &ed = N[n].next[i], y = N[prev].next[i];
                if (ed == -1) ed = y;
                else {
                    N[ed].back = y;
                    (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                        = N[y].end;
                    N[ed].nmatches += N[y].nmatches;
                    q.push(ed);
                }
            }
        }
    }
    vi find(string word) {
        int n = 0;
        vi res; // ll count = 0;
        for (char c : word) {
            n = N[n].next[c - first];
            res.push_back(N[n].end);
            // count += N[n].nmatches;
        }
        return res;
    }
    vector<vi> findAll(vector<string>& pat, string word) {
        vi r = find(word);
        vector<vi> res(sz(word));
        rep(i,0,sz(word)) {
            int ind = r[i];
            while (ind != -1) {
                res[i - sz(pat[ind]) + 1].push_back(ind);
                ind = backp[ind];
            }
        }
        return res;
    }
};
```